ИНСТРУКЦИЯ ПО УСТАНОВКЕ SWI-PROLOG И ОСНОВАМ ПРАКТИЧЕСКОЙ РАБОТЫ В СРЕДЕ SWI-PROLOG

УСТАНОВКА SWI-PROLOG

1. Для начала установки перейти на страницу http://www.swi-prolog.org/download/stable .
2. Для ОС Windows выбрать ссылку https://www.swi-prolog.org/download/stable/bin/swipl-9.2.9-1.x64.exe.envelope.
3. Скачайте файл по ссылке Download swipl-9.2.9-1.x64.exe.
4. Осуществите установку SWI-Prolog, выполнив файл swipl-9.2.9-1.x64.exe с параметрами по умолчанию.
5. Система SWI-Prolog готова к работе.

ЭЛЕМЕНТЫ ПРАКТИЧЕСКОЙ РАБОТЫ В СРЕДЕ SWI-Prolog

1. Ввод и компиляция программы

Рассмотрим простейшую Пролог-программу:

% *Факты*
отец('Иван','Сергей'). % Иван — отец Сергея
отец('Иван','Анна'). % Иван — отец Анны
мама('Мария','Сергей'). % Мария — мать Сергея

% *Правило*
/* О и М являются родителями ребенка Р,
если О является отцом Р, а М является мамой Р */
родители(О,М,Р): – отец(О,Р), мама(М,Р).

Приступим к созданию и выполнению приведенной выше программы с использованием SWI-Prolog. Для этого необходимо выполнить несколько шагов.

Запустите SWI-Prolog. Откроется сессия работы с интерпретатором SWI-Prolog и будут выведены сообщения, изображенные на рис. 1.

Рис. 1. – Окно SWI-Prolog

Выберите **File > New .** Откроется окно создания файла, в поле **Имя файла** введите имя файла, в котором будет храниться исходный код программы, например **parents** и нажмите кнопку **Сохранить**. Файлы с исходными кодами Пролог программ должны иметь тип **.pl**.

В файл **parents.pl,** открытый в окне редактора, введите текст рассмотренной выше программы. Результаты ввода изображены на рис. 2.

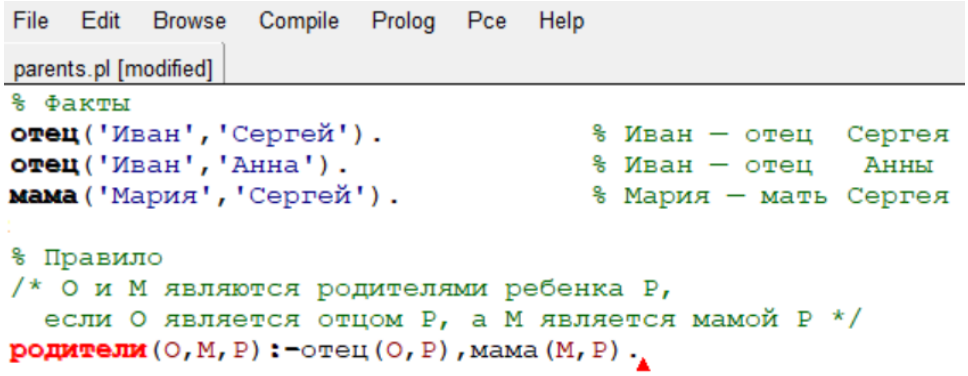

Рис. 2. – Окно редактора с Пролог программой

Сохраните файл, выбрав в окне редактора **File > Save Buffer.**

Чтобы доказать (выполнить) то или иное целевое утверждение, программу следует скомпилировать. Для компиляции в окне редактора можно выбрать **Compile->Compile Buffer**. Если в процессе компиляции утверждений программы будут обнаружены синтаксические ошибки, то выводятся соответствующие сообщения, ссылающиеся на номера строк программы. Номер строки программы, в которой находится курсор, отображается в нижнем правом углу окна редактора.

2. Выполнение запросов

Чтобы проверить работу программы вводите последовательно в окне интерпретатора запросы (целевые утверждения). При этом символы “**?-**“ вводить не требуется. Например, введем запрос

отец(‘Иван’, P). % Кто дети Ивана?

SWI-Prolog выполнит интерпретацию этого целевого утверждения и в результате сопоставления с предикатами загруженной базы данных выведет первое значение переменной **P** :

P = ‘Сергей’

Для получения следующего значения переменной **P** введите символ “**;**” и нажмите **Enter**. SWI-Prolog выведет второе значение

P = ‘Анна’.

Чтобы проверить работу правила программы введите запрос (кто родители Сергея?):

родители(O,M,’Сергей’).

В результате доказательства этого целевого утверждения будут выведены следующие значения переменных **O** и **M**, при которых оно справедливо:

Иван
Мария

Целевые утверждения можно вводить не только во время сеанса интерпретации, но и включать в исходный код программы во время её создания. В этом случае целевое утверждение должно начинаться с символов “**?-**“ (или “**:-**“). Добавьте в конец программы в окне редактора следующие строки:

% Целевое утверждение (запрос)
?-родители(O,M,’Сергей’),nl,write(O),nl,write(M).

Здесь **write** —это встроенный предикат вывода, а **nl** — встроенный предикат, обеспечивающий перевод курсора на новую строку.

Сохраните файл и выполните заново компиляцию программы. Теперь SWI-Prolog начнет сразу доказательство целевого утверждения, содержащегося в программе. При этом условия (предикаты) целевого утверждения будут выполняться в порядке их записи.

Сначала SWI-Prolog выполнит поиск возможных вариантов сопоставлений для предиката **родители(О,М,'Сергей').** При этом переменные **О** и **М** будут конкретизированы и получат значения: '**Иван'** и **'Мария'**. Предикаты **write** обеспечат вывод этих значений в окно интерпретатора.

В примере программы использовались имена предикатов и констант, набранные литерами русского алфавита. SWI-Prolog поддерживает обработку таких имен. Однако в ряде случаев редактор такие имена обрабатывает некорректно. Кроме этого, ухудшается переносимость программ. Поэтому при разработке программ лучше использовать литеры английского алфавита.

ВОЗМОЖНОСТИ ИНТЕРПРЕТАТОРА SWI-PROLOG

1. Помощь

В процессе работы со SWI-интерпретатором доступна "on-line"помощь. Для этого можно использовать следующие встроенные предикаты:

- **apropos(Образец) —** отображает все предикаты, функции или разделы помощи, которые содержат в своем имени подстроку **Образец**;
- **explain(Объект) —** выводит текст с объяснением любого объекта данных Пролога.

Например, если в окне консоли ввести запрос **apropos(assert)**, то будут выведены все предикаты, которые содержат в своем описании подстроку **assert** (рис.3).

```
?- apropos(assert)
|    .
% SWI assert/1  Assert a clause (fact or rule) into the database.
% SWI assert/2  Equivalent to asserta/1, assertz/1, assert/1, but in addition
unifies Reference with a handle to the asserted ...
% ISO asserta/1  Assert a clause (fact or rule) into the database.
% ISO assertz/1  Assert a clause (fact or rule) into the database.
% SWI asserta/2 Equivalent to asserta/1, assertz/1, assert/1, but in addition
unifies Reference with a handle to the asserted ...
```

Рис.3. — Результаты выполнения запроса **apropos(assert)**

Если ввести в окне консоли **explain(assert)**, то будет выведен текст с объяснением встроенного предиката **assert** (рис.4).

```
?- explain(assert).
% assert is an atom
%     Referenced from 3-th clause of doc_words:synonym/3 at c:/program
        files/swipl/library/ext/pldoc/pldoc/doc_words.pl:76
% system:assert/1 is a built-in meta predicate
%     Referenced from 1-th clause of prolog_debug:debug/2 at c:/program
        files/swipl/library/debug.pl:149
….
% system:assert/2 is a built-in meta predicate
true.
```

Рис. 4. — Результаты выполнения запроса **explain(assert)**

2. История команд

SWI-Prolog хранит в специальном списке перечень запросов (команд), которые были введены ранее. Такой список называют историей команд. Чтобы включить историю, запоминающую последние 50 команд, поместите следующую команду в startup файл запуска (или выполните запрос):

?- set_prolog_flag(history, 50).

Таблица 1 содержит перечень команд, которые позволяют выбирать и повторно выполнять ранее введенные запросы. Также перемещаться по списку истории команд можно с помощью клавиш со стрелками вверх/вниз.

Таблица 1 —Доступ к истории команд

Команда	Назначение команды
!!.	Повторить последний запрос
!*номер*.	Повторить запрос с заданным номером
!*подстрока*.	Повторить последний запрос, начинающийся заданной подстрокой
h.	Показать историю команд
!h.	Показать эту таблицу

Например, если в окне консоли ввести команду “**h.**”, то получим список ранее введенных запросов (рис. 5).

```
1   отец(иван,Р).
2   listing.
3   отец('Иван','Сергей').
4   отец('Иван',Р).
5   отец(иван,Р).
6   отец('Иван',Р).
7   родители(О,М,'Сергей').
```

Рис. 5. — История команд

3. Отладка программы

Двумя основными средствами проверки работы программы в Прологе являются *трассировка и контрольные точки*.

Для включения режима трассировки используется предикат **trace.** Для её отключения — предикат **notrace.** Трассировка позволяет пользователю наблюдать за ходом выполнения программы во время её работы. Предикат **trace** вводится непосредственно перед проверяемым участком программы или запросом, а предикат **notrace** — сразу после него.

SWI-Prolog позволяет пользователю выбирать режим трассировки, при котором информация о ходе трассировки выдается в четырех стандартных точках (портах) обработки утверждения:

Call (вызов) — точка вызова доказываемого утверждения;

Exit (выход) — точка успешного выхода при доказательстве утверждения;

Redo (передоказать) — точка повторного доказательства утверждения;

Fail (неудача) — точка с состоянием неудачи при доказательстве утверждения.

В дополнение к этим точкам SWI-Prolog предлагает точку (порт) **unify**, которая позволяет пользователю инспектировать результаты после сопоставления (унификации) головы утверждения. Также имеется порт **exception**, который показывает исключительные ситуации, вызванные предикатом **throw**.

Предикат **trace/0** (цифра после знака "/" обозначает количество аргументов предиката), введенный в окне консоли, обеспечивает трассировку следующего за ним запроса (целевого утверждения). В ходе трассировки отображается имя порта, текущая глубина рекурсивного вызова и очередная цель.

Рассмотрим пример трассировки. Для этого добавим в текст программы, изображенной на рис.4, предикат **дети(Отец)**, обеспечивающий вывод на экран имен всех детей для заданного отца:

дети(Отец):-
отец(Отец,Ребенок),nl,write(Ребенок),fail;
true.

Сохраним файл и заново его скомпилируем. Введем последовательно два запроса: **trace** и **дети('Иван').** Теперь при доказательстве утверждения **дети('Иван')** при каждом нажатии клавиши **Enter** в окне консоли будут построчно выводиться сообщения трассировщика (рис. 6).

```
?-
|    trace.
true.

[trace]  ?- дети('Иван').
   Call: (12) дети('Иван') ? creep
   Call: (13) отец('Иван', _33370) ? creep
   Exit: (13) отец('Иван', 'Сергей') ? creep
   Call: (13) nl ? creep

   Exit: (13) nl ? creep
   Call: (13) write('Сергей') ? creep
Сергей
   Exit: (13) write('Сергей') ? creep
   Call: (13) fail ? creep
   Fail: (13) fail ? creep
   Redo: (13) отец('Иван', _33370) ? creep
   Exit: (13) отец('Иван', 'Анна') ? creep
```

```
   Call: (13) nl ? creep

   Exit: (13) nl ? creep
   Call: (13) write('Анна') ? creep
Анна
  Exit: (13) write('Анна') ? creep
   Call: (13) fail ? creep
   Fail: (13) fail ? creep
   Redo: (12) дети('Иван') ? creep
   Call: (13) true ? creep
   Exit: (13) true ? creep
   Exit: (12) дети('Иван') ? creep
true.

[trace]  ?- notrace.
true.
```

Рис. 6. — Трассировка запроса

При этом в стандартных точках выдачи информации после знака “?” можно вводить команды трассировщика. Команды трассировщика обозначаются одной литерой. В таблице 2 приведены некоторые команды трассировщика SWI-Prolog.

Таблица 2 — Команды трассировщика SWI-Prolog

Команда	Название	Назначение команды
A	Alternativies	Показать все цели, имеющие альтернативы.
L	Listing	Отобразить список утверждений с помощью предиката listing.
a	abort	Вернуться на верхний уровень интерпретатора.
b	break	Рекурсивно запускает интерпретатор с верхнего уровня.
c	creep	Продолжить выполнение до следующего порта.
e	exit	Завершить работу интерпретатора.
f	fail	Установить результат “неудача” при доказательстве текущей цели.
g	goals	Показать список дочерних целей (стек выполнения)
h	help	Отобразить список допустимых команд трассировщика
i	ignore	Игнорировать текущую цель
l	leap	Продолжить выполнение до следующей контрольной точки.
n	no debug	Продолжить выполнение не в режиме отладки
r	retry	Возврат к call порту текущей цели
s	skip	Продолжить выполнение до следующего порта цели, пропустив все дочерние цели
u	up	Продолжить выполнение и сделать остановку при заходе в порт родительской цели (т.о. пропускается текущая цель и все вызовы дочерних целей).
+	Spy	Установить контрольную точку для текущего предиката
-	No Spy	Удалить контрольную точку для текущего предиката
/	find	Выполнить поиск порта. После ‘/’ следует указать имя порта (можно сокращенно). Например, /f — поиск порта fail; /c — поиск порта вызова; /a — поиск любого порта

В процессе отладки иногда возникает необходимость приостановить работу программы при доказательстве очередной цели (целевого

утверждения). Для этого применяют контрольные точки. Остановка в контрольной точке позволяет проверить состояние параметров и, при необходимости, воздействовать на дальнейший процесс доказательства. При достижении контрольной точки запускается трассировщик, если система SWI-Prolog находится в режиме отладки (включить режим отладки можно с помощью предиката **debug**).

Для установки контрольных точек на предикатах, удовлетворяющих заданной спецификации применяется вызов **spy(***спецификация предиката***)**, для её удаления — **nospy(***спецификация предиката)***.** Например, если мы хотим установить контрольную точку на любом предикате **отец,** то следует применить вызов **spy(отец/2)** или **spy(отец).** Пример установки и удаления контрольных точек при работе с интерпретатором SWI-Prolog изображен на рис. 7.

```
?- spy(отец/2).
% Spy point on отец/2
true.

[debug]  ?- дети('Иван').
   Call: (13) отец('Иван', _12624) ? creep
   Exit: (13) отец('Иван', 'Сергей') ? creep
   Call: (13) nl ? creep

   Exit: (13) nl ? creep
   Call: (13) write('Сергей') ? creep
Сергей
   Exit: (13) write('Сергей') ? creep
   Call: (13) fail ? creep
   Fail: (13) fail ? creep
   Redo: (13) отец('Иван', _12624) ? creep
   Exit: (13) отец('Иван', 'Анна') ? creep
   Call: (13) nl ? creep

   Exit: (13) nl ? creep
   Call: (13) write('Анна') ? creep
Анна
   Exit: (13) write('Анна') ? creep
   Call: (13) fail ? creep
   Fail: (13) fail ? creep
   Redo: (12) дети('Иван') ? creep
   Call: (13) true ? creep
   Exit: (13) true ? creep
   Exit: (12) дети('Иван') ? creep
true.

[trace]  ?- debugging.
% Debug mode is off
% No exception traps
true.

[trace]  ?- nospy(отец/2).
% Spy point removed from отец/2
true.
```

Рис.7. — Установка и удаление контрольных точек

Предикат **debugging** выводит в текущий выходной поток сведения о режиме отладки, установленных контрольных точках и трассируемых предикатах. По умолчанию режим отладки отключен. Однако система переходит в режим отладки при первом вызове предикатов: **debug, trace** или **spy**. Для отключения режима отладки применяется предикат **nodebug**.